App open ads

Page Summary

Select platform: Android (beta) NEW (/admob/android/next-gen/app-open) Android (/admob/android/app-open) iOS (/admob/ios/app-open) Unity (/admob/unity/app-open) Flutter (/admob/flutter/app-open)

This guide is intended for publishers integrating app open ads using Google Mobile Ads SDK.

App open ads are a special ad format intended for publishers wishing to monetize their app load screens. App open ads can be closed at any time, and are designed to be shown when your users bring your app to the foreground.

Note: Specific format may vary by region.

App open ads automatically show a small branding area so users know they're in your app. Here is an example of what an app open ad looks like:

Prerequisites

Before you continue, set up Google Mobile Ads SDK (/admob/android/quick-start).

Always test with test ads

When building and testing your apps, make sure you use test ads rather than live, production ads. Failure to do so can lead to suspension of your account.

The easiest way to load test ads is to use our dedicated test ad unit ID for app open ads:

```
ca-app-pub-3940256099942544/9257395921
```

It's been specially configured to return test ads for every request, and you're free to use it in your own apps while coding, testing, and debugging. Just make sure you replace it with your own ad unit ID before publishing your app.

For more information about how Google Mobile Ads SDK test ads work, see Enable test ads (/admob/android/test-ads).

Extend the Application class

Create a new class that extends the **`Application`** class. This provides a lifecycle-aware way to manage ads that are tied to the application's state rather than a single **`Activity`**:

Java (#java)Kotlin (#kotlin)

```
class MyApplication :
  Application(), Application.ActivityLifecycleCallbacks, DefaultLifecycleObserv

  private lateinit var appOpenAdManager: AppOpenAdManager
  private var currentActivity: Activity? = null

  override fun onCreate() {
    super<Application>.onCreate()
    registerActivityLifecycleCallbacks(this)

    ProcessLifecycleOwner.get().lifecycle.addObserver(this)
    appOpenAdManager = AppOpenAdManager()
  }
```

ıple/app/src/main/java/com/google/android/gms/example/appopenexample/MyApplication.kt#L22-L35)

Next, add the following code to your **AndroidManifest.xml**:

```
<!-- TODO: Update to reference your actual package name. -->
<application
    android:name="com.google.android.gms.example.appopendemo.MyApplication" ...>
...
</application>
```

Implement your utility component

Your ad should show quickly, so it's best to load your ad before you need to display it. That way, you'll have an ad ready to go as soon as your user enters into your app.

Implement a utility component **AppOpenAdManager** to encapsulate the work related to loading and showing App Open ads:

Java (#java)Kotlin (#kotlin)

```
private inner class AppOpenAdManager {

  private var appOpenAd: AppOpenAd? = null
  private var isLoadingAd = false
  var isShowingAd = false

  /** Keep track of the time an app open ad is loaded to ensure you don't show
  private var loadTime: Long = 0
```

e/app/src/main/java/com/google/android/gms/example/appopenexample/MyApplication.kt#L111-L121)

To use the **AppOpenAdManager**, call the public wrapper methods on the singleton **MyApplication** instance. The **Application** class interfaces with the rest of the code, delegating the work of loading and showing the ad to the manager.

Load an ad

The next step is to fill out the `loadAd()` method and handle the ad load callbacks.

Java (#java)Kotlin (#kotlin)

```
AppOpenAd.load(
  context,
  "AD_UNIT_ID",
  AdRequest.Builder().build(),
  object : AppOpenAdLoadCallback() {
    override fun onAdLoaded(ad: AppOpenAd) {
      // Called when an app open ad has loaded.
      Log.d(LOG_TAG, "App open ad loaded.")

      appOpenAd = ad
      isLoadingAd = false
      loadTime = Date().time
    }

    override fun onAdFailedToLoad(loadAdError: LoadAdError) {
      // Called when an app open ad has failed to load.
      Log.d(LOG_TAG, "App open ad failed to load with error: " + loadAdError.me

      isLoadingAd = false
    }
  },
)
```

e/app/src/main/java/com/google/android/gms/example/appopenexample/MyApplication.kt#L136-L159)

Replace ***AD_UNIT_ID*** with your own ad unit ID.

Show the ad

The most common app open implementation is to attempt to show an app open ad near app launch, start app content if the ad isn't ready, and preload another ad for the next app open opportunity. See App open ad guidance (//support.google.com/admob/answer/9341964) for implementation examples.

The following code shows and subsequently reloads an ad:

Java (#java)Kotlin (#kotlin)

```
fun showAdIfAvailable(activity: Activity, onShowAdCompleteListener: OnShowAdCom
  // If the app open ad is already showing, do not show the ad again.
  if (isShowingAd) {
    Log.d(TAG, "The app open ad is already showing.")
    return
  }

  // If the app open ad is not available yet, invoke the callback then load the
  if (appOpenAd == null) {
    Log.d(TAG, "The app open ad is not ready yet.")
    onShowAdCompleteListener.onShowAdComplete()
    // Load an ad.
    return
  }

  isShowingAd = true
  appOpenAd?.show(activity)
}
```

nced/APIDemo/app/src/main/java/com/google/android/gms/snippets/AppOpenAdSnippets.kt#L51-L72)

Set the FullScreenContentCallback

The **`FullScreenContentCallback`** handles events related to displaying your **`AppOpenAd`**. Before showing **`AppOpenAd`**, make sure to set the callback:

Java (#java)Kotlin (#kotlin)

```
appOpenAd?.fullScreenContentCallback =
  object : FullScreenContentCallback() {
    override fun onAdDismissedFullScreenContent() {
      // Called when full screen content is dismissed.
      Log.d(TAG, "Ad dismissed fullscreen content.")
      // Don't forget to set the ad reference to null so you
      // don't show the ad a second time.
      appOpenAd = null
      isShowingAd = false

      onShowAdCompleteListener.onShowAdComplete()
      // Load an ad.
    }
```

```
    override fun onAdFailedToShowFullScreenContent(adError: AdError) {
      // Called when full screen content failed to show.
      Log.d(TAG, adError.message)
      // Don't forget to set the ad reference to null so you
      // don't show the ad a second time.
      appOpenAd = null
      isShowingAd = false

      onShowAdCompleteListener.onShowAdComplete()
      // Load an ad.
    }

    override fun onAdShowedFullScreenContent() {
      Log.d(TAG, "Ad showed fullscreen content.")
    }

    override fun onAdImpression() {
      // Called when an impression is recorded for an ad.
      Log.d(TAG, "The ad recorded an impression.")
    }

    override fun onAdClicked() {
      // Called when ad is clicked.
      Log.d(TAG, "The ad was clicked.")
    }
  }
```

ced/APIDemo/app/src/main/java/com/google/android/gms/snippets/AppOpenAdSnippets.kt#L80-L119)

Key Point: If a user returns to your app after having left it by clicking on an app open ad, it makes sure they're not presented with another app open ad.

Consider ad expiration

Key Point: App open ads will time out after four hours. Ads rendered more than four hours after request time will no longer be valid and may not earn revenue.

To make sure you don't show an expired ad, add a method to the `AppOpenAdManager` that checks how long it has been since your ad reference loaded. Then, use that method to check if the ad is still valid.

Java (#java)Kotlin

```
/** Check if ad was loaded more than n hours ago. */
private fun wasLoadTimeLessThanNHoursAgo(numHours: Long): Boolean {
  val dateDifference: Long = Date().time - loadTime
  val numMilliSecondsPerHour: Long = 3600000
  return dateDifference < numMilliSecondsPerHour * numHours
}

/** Check if ad exists and can be shown. */
private fun isAdAvailable(): Boolean {
  // For time interval details, see: https://support.google.com/admob/answer/93
  return appOpenAd != null && wasLoadTimeLessThanNHoursAgo(4)
}
```

e/app/src/main/java/com/google/android/gms/example/appopenexample/MyApplication.kt#L164-L176)

Keep track of current activity

To show the ad, you'll need an `Activity` context. To keep track of the most current activity being used, register for and implement the **`Application.ActivityLifecycleCallbacks`** (//developer.android.com/reference/android/app/Application.ActivityLifecycleCallbacks).

Java (#java)Kotlin (#kotlin)

```
override fun onActivityCreated(activity: Activity, savedInstanceState: Bundle?)

override fun onActivityStarted(activity: Activity) {
  // An ad activity is started when an ad is showing, which could be AdActivity
  // SDK or another activity class implemented by a third party mediation partn
  // currentActivity only when an ad is not showing will ensure it is not an ad
  // one that shows the ad.
  if (!appOpenAdManager.isShowingAd) {
    currentActivity = activity
  }
}

override fun onActivityResumed(activity: Activity) {}

override fun onActivityPaused(activity: Activity) {}
```

```
override fun onActivityStopped(activity: Activity) {}

override fun onActivitySaveInstanceState(activity: Activity, outState: Bundle)

override fun onActivityDestroyed(activity: Activity) {}
```

ıple/app/src/main/java/com/google/android/gms/example/appopenexample/MyApplication.kt#L54-L75)

`registerActivityLifecycleCallbacks` (//developer.android.com/reference/android/app/Application#registerActivityLifecycleCallbacks(android.app.Application.ActivityLifecycleCallbacks)) lets you listen for all `Activity` events. By listening for when activities are started and destroyed, you can keep track of a reference to the current `Activity`, which you then will use in presenting your app open ad.

Listen for app foregrounding events

To listen for app foreground events, do the following steps:

Add the libraries to your gradle file

To be notified of app foregrounding events, you need to register a `DefaultLifecycleObserver`. Add its dependency to your app-level build file:

Kotlin Groovy

build.gradle.kts build.gradle

```
dependencies {
  implementation("com.google.android.gms:play-services-ads:25.0.0")
  implementation("androidx.lifecycle:lifecycle-process:2.8.3")
}
```

Implement the lifecycle observer interface

You can listen for foregrounding events by implementing the `DefaultLifecycleObserver` interface.

Implement the `onStart()` to show the app open ad.

```
override fun onStart(owner: LifecycleOwner) {
  super.onStart(owner)
  currentActivity?.let {
    // Show the ad (if available) when the app moves to foreground.
    appOpenAdManager.showAdIfAvailable(it)
  }
}
```

ıple/app/src/main/java/com/google/android/gms/example/appopenexample/MyApplication.kt#L42-L49)

Cold starts and loading screens

The documentation thus far assumes that you only show app open ads when users foreground your app when it is suspended in memory. "Cold starts" occur when your app is launched but was not previously suspended in memory.

An example of a cold start is when a user opens your app for the first time. With cold starts, you won't have a previously loaded app open ad that's ready to be shown right away. The delay between when you request an ad and receive an ad back can create a situation where users are able to briefly use your app before being surprised by an out of context ad. This should be avoided because it is a bad user experience.

The preferred way to use app open ads on cold starts is to use a loading screen to load your game or app assets, and to only show the ad from the loading screen. If your app has completed loading and has sent the user to the main content of your app, don't show the ad.

Key Point: To continue loading app assets while the app open ad is being displayed, always load assets in a background thread.

Best practices

App open ads help you monetize your app's loading screen, when the app first launches and during app switches, but it's important to keep best practices in mind so that your users enjoy using your app. It's best to:

- Show your first app open ad after your users have used your app a few times.
- Show app open ads during times when your users would otherwise be waiting for your app to load.
- If you have a loading screen under the app open ad, and your loading screen completes loading before the ad is dismissed, you may want to dismiss your loading screen in the `onAdDismissedFullScreenContent()` method.

Examples on GitHub

- App Open ads example: Java (//github.com/googleads/googleads-mobile-android-examples/tree/main/java/admob/AppOpenExample) | Kotlin (//github.com/googleads/googleads-mobile-android-examples/tree/main/kotlin/admob/AppOpenExample)

Next steps

Explore the following topics:

- User privacy (/admob/android/privacy)
- Optimized SDK initialization and ad loading (beta) (/admob/android/optimize-initialization)

Last updated 2026-02-27 UTC.